ASTRAFORGE AEROSPACE · YNOV BACHELOR 2

ORBITAL COMMAND

Jour 1 — TypeScript Mission Models

DYLAN · MERCREDI 2 SEPTEMBRE 2026 · 4 HEURES · 20 % THÉORIE · 80 % PRATIQUE

BUTS DU COURS

- Laisser TypeScript inférer les types simples
- Modéliser un domaine avec type et interface
- Utiliser une union discriminante
- Construire un générique utile
- Démarrer React + TypeScript en mode strict

SÉQUENCE 1 · BRIEFING

D'abord, le projet fil rouge

Une démo live d'Orbital Command — version complète, de A à Z

Avant le premier type, on regarde **la destination**.

- flotte, missions, télémétrie, alertes
- le parcours opérateur de bout en bout
- ce que vous construirez sur les 10 jours

Ensuite je prends la main : démo live du produit fini.
Pas de slides pendant la démo.

SÉQUENCE 1 · BRIEFING

Démo live

Orbital Command — centre de contrôle opérationnel

Observez le parcours. On reviendra ensuite au premier modèle TypeScript.

SÉQUENCE 1 · BRIEFING

ORBITAL COMMAND

Jour 1 — TypeScript Mission Models

Avant de lancer une fusée, il faut empêcher les données de mentir.

SÉQUENCE 1 · BRIEFING

La mission du module

Construire un centre de contrôle de missions spatiales

- gérer une flotte de véhicules
- planifier des missions
- recevoir de la télémétrie
- déclencher et traiter des alertes
- livrer une application testée et déployée

AstraForge a récupéré un système critique… écrit trop vite

Le manifeste contient :

- des identifiants parfois numériques, parfois textuels
- des unités mélangées
- des statuts impossibles
- des champs absents
- plusieurs `any` particulièrement suspects

Votre équipe reprend le contrôle.

SÉQUENCE 1 · BRIEFING

Objectifs du jour

À la fin de la séance, vous saurez

- laisser TypeScript inférer les types simples
- modéliser un domaine avec `type` et `interface`
- utiliser une union discriminante
- construire un générique utile
- démarrer une application React + TypeScript stricte

SÉQUENCE 1 · BRIEFING

Plan de vol

1. Corrupted Flight Manifest
2. Fleet Registry Types
3. Vehicle Identification
4. Telemetry Buffer `<T>`
5. Initialize Orbital Command
6. Revue et ticket de sortie

TypeScript vérifie avant le décollage

JavaScript exécute. TypeScript analyse.

fuel-percent.ts

```ts
const fuelPercent = 82
fuelPercent.toUpperCase()
// Erreur détectée avant l'exécution
```

TypeScript ne remplace pas les tests : il élimine une famille d'erreurs plus tôt.

Inférer ou annoter ?

infer-or-annotate.ts

```ts
const callsign = "AQUILA-7"      // inférence
let fuelPercent: number = 82     // annotation utile
```

Laisser l'inférence quand l'intention est évidente. Annoter les frontières : paramètres, retours publics, données externes.

SÉQUENCE 2 · TP 1

Corrupted Flight Manifest

Réparer un manifeste volontairement corrompu

- identifier au moins huit incohérences
- remplacer les `any`
- choisir une unité unique par mesure
- corriger les fonctions jusqu'à compilation stricte

Contrainte – interdiction d'utiliser `as any` ou `@ts-ignore`

SÉQUENCE 2 · INCIDENT

Message du contrôle

Le carburant de Titan vaut "0.78".
Est-ce 0,78 %, 78 % ou une chaîne inutilisable ?

Choisir une représentation interne explicite et documenter l'unité.

SÉQUENCE 2 · DÉBRIEF

Débrief du TP 1

- quelles erreurs TypeScript a-t-il empêchées ?
- quelles ambiguïtés nécessitaient une décision métier ?
- à quel endroit une conversion devait-elle avoir lieu ?

Un bon type décrit une décision, pas seulement une forme.

type et interface

La cohérence compte davantage que le duel de syntaxe

type-interface.ts

```ts
interface VehicleBase {
  readonly id: string
  name: string
  status: VehicleStatus
}

type VehicleStatus = "grounded" | "ready" | "in-flight"
```

Optionnel ne veut pas dire inconnu

optional.ts

```ts
interface Mission {
  id: string
  launchAt?: string
}
```

`launchAt?` signifie : la propriété peut être absente. Si la donnée existe mais n'est pas encore comprise, son type doit le dire autrement.

readonly protège l'identité

readonly.ts

```ts
interface Vehicle {
  readonly id: string
  name: string
}
```

L'identifiant d'Aquila ne doit pas changer lorsqu'on modifie son statut.

SÉQUENCE 3 · TP 2

Fleet Registry Types

Typer Aquila, Titan et Hermes

- Aquila, véhicule habité
- Titan, lanceur cargo
- Hermes, navette autonome

Livrable : un tableau `vehicles` valide sans `any`, avec propriétés communes et spécialisées.

Revue express du registre

- les identifiants sont immuables
- les statuts sont limités à des valeurs autorisées
- les propriétés réellement absentes sont optionnelles
- les unités apparaissent dans les noms ou la documentation
- aucun type n'est plus large que nécessaire

Un champ pour identifier chaque véhicule

kind est le discriminant

discriminated-union.ts

```ts
type Vehicle =
  | { kind: "crewed"; crewCapacity: number }
  | { kind: "cargo"; payloadKg: number }
  | { kind: "autonomous"; autonomyLevel: number }
```

Le discriminant permet à TypeScript de réduire le type.

Narrowing en action

narrowing.ts

```ts
function describeVehicle(vehicle: Vehicle) {
  if (vehicle.kind === "cargo") {
    return `${vehicle.payloadKg} kg de charge utile`
  }
}
```

Dans ce bloc, TypeScript sait que `vehicle` est un cargo.

Rendre l'oubli impossible

assert-never.ts

```ts
function assertNever(value: never): never {
  throw new Error(`Cas non traité: ${value}`)
}
```

Une branche `default` avec `never` transforme l'ajout d'un véhicule en rappel de mise à jour.

SÉQUENCE 4 · TP 3

Vehicle Identification

Créer getVehicleCapability(vehicle)

- un texte propre à chaque catégorie
- un narrowing sans assertion forcée
- un traitement exhaustif
- un quatrième véhicule ajouté pour éprouver le modèle

SÉQUENCE 4 · BOSS FIGHT

Le véhicule hybride JANUS

Il transporte du fret et un équipage

Faut-il :

- ajouter une nouvelle catégorie
- composer des capacités
- rendre plusieurs propriétés optionnelles ?

Justifiez la solution qui rend les états impossibles difficiles à représenter.

Débrief du TP 3

- une union modélise des variantes fermées
- un discriminant rend le code lisible
- `never` signale les branches oubliées
- une avalanche de propriétés optionnelles masque souvent un mauvais modèle

Un même comportement, plusieurs données

telemetry-point.ts

```ts
type TelemetryPoint<T> = {
  timestamp: number
  value: T
}
```

`T` garde la structure réutilisable sans perdre l'information du type transporté.

SÉQUENCE 5 · TP 4

Telemetry Buffer T

Un buffer borné, un seul type à la fois

- accepte des points d'un type donné
- conserve seulement les N derniers
- retourne le dernier relevé ou `undefined`
- ne mélange jamais carburant et température

SÉQUENCE 5 · RÉSISTANCE

Test de résistance du buffer

1. buffer vide
2. un seul point
3. dépassement de la capacité
4. valeurs objet
5. tentative d'insertion d'un type incompatible

Le cinquième scénario doit échouer à la compilation.

Débrief du TP 4

Le générique est-il utile ou seulement décoratif ?

Bon signal : le code appelant récupère précisément le type qu'il a inséré, sans conversion.

Mode strict — le simulateur sans filet

Le projet doit refuser :

- les paramètres implicitement `any`
- les accès potentiellement indéfinis non traités
- les retours incohérents
- les imports et variables inutilisés selon la configuration choisie

TSX — des données typées deviennent une interface

vehicle-card.tsx

```tsx
type VehicleCardProps = {
  vehicle: Vehicle
}

export function VehicleCard({ vehicle }: VehicleCardProps) {
  return <article>{vehicle.name}</article>
}
```

Les props constituent le premier contrat entre le modèle et l'interface.

SÉQUENCE 6 · TP 5

Initialize Orbital Command

1. créer le projet Vite React + TypeScript
2. activer et vérifier le mode strict
3. intégrer les modèles de flotte
4. afficher Aquila, Titan et Hermes
5. montrer leur capacité spécifique

SÉQUENCE 6 · MINI-DÉFI

Statut visuel exhaustif

getStatusLabel + un style par statut

GROUNDED | READY | IN-FLIGHT | MAINTENANCE

L'ajout d'un statut doit provoquer une erreur tant que son affichage n'est pas défini.

Definition of Done

- `npm run dev` démarre
- le type-check passe
- le mode strict est actif
- aucun `any` non justifié
- trois véhicules s'affichent
- chaque variante montre sa propriété spécifique
- le buffer conserve une taille bornée

SÉQUENCE 7 · CLÔTURE

Revue en binômes

Chaque binôme vérifie chez l'autre :

1. un type trop large
2. un cas non exhaustif
3. une unité ambiguë
4. un bon choix de modélisation à conserver

Commit de mission

Tag attendu — session-01

commit.txt

```
feat(session-01): model fleet and initialize orbital command
```

SÉQUENCE 7 · TICKET DE SORTIE

Répondez en trois lignes

1. une erreur que TypeScript a détectée pour vous
2. un état impossible supprimé par votre modèle
3. un point encore flou à éclaircir

SÉQUENCE 7 · SUITE

React Flight Deck

Demain, les modèles deviennent une véritable interface

- composants
- props
- état local
- événements
- Tailwind CSS et responsive

Les données sont fiables. Il faut maintenant construire le cockpit.